[High-Level] Infrastructure and Workflow for Smart-Stock

TITLE: INFRASTRACTURE AND WORKFLOW

Project: SMART-STOCK: A Web-Based Intelligent Inventory and Supply Management System for Hardware Stores

INFRASTRACTURE PLAN:

ENVIRONMENTS OVERVIEW:

Concern	Local EXISTS	Test EXISTS	Staging EXISTS	Production EXISTS
App hosting	Own laptop via NGROK	Render free tier	Render Free tier	Render free tier
Database	Own laptop via Session Pooler	Supabase free tier	Supabase free tier	Supabase free tier
Deployed from	Working copy	Github main branch via autodeploy	GitHub staging branch via manual trigger	GitHub release tag via approval gate
Who may deploy	developer / development team	Automated via Render on merge to main	Development Team and QA	Development Team & Project Manager
Data	Live Supabase shared data	Live Supabase shared data	Seeded resettable data	Final production test dataset
Monthly cost	$0	$0	$0	$0

APP HOSTING:

Provider	Render free tier
Runtime	PHP 8.3-FPM + Nginx + Supervisor in single Docker container
Framework	Laravel 12
Frontend Build	Vite 7 + Tailwind CSS 4
Test URL	https://smart-stock-4lav.onrender.com
Sleeps When Idle	Yes, spins down after 15 minutes of inactivity
Scaling	Single instance (free tier limitation)

DATABASE HOSTING:

Provider	Supabase free tier

Engine	PostgreSQL 15 plus
Size	1 gb storage limit
Connection Type	pgsql with SSLMODE require
App-Level Backup	JSON dump to storage app private backups via backup run command, downloadable in Admin Backups page
Testing Database	SQLite memory only for phpunit, not primary
Backups	Supabase automated daily snapshots with 7-day retention

OTHER SERVICES:

Service	Used for	Notes
CI/CD Pipeline	Automated testing and deployment to Render	Render autoDeploy from Dockerfile, no GitHub Actions minutes needed
DNS + TLS	Secure Test URL (https://)	Provider-managed certificate via Render
Config Storage	Environment variables	Stored in Render dashboard, not hardcoded, ENV file excluded from Docker image
Secrets Storage	APP KEY, DB HOST, DB PASSWORD	Stored in Render Environment vault with sync false, never logged

DEPLOYMENT / WORKFLOW PROCESS

SPRINT RELEASE:

Every time code is pushed to the DEV branch, a Pull Request (DEV to MAIN) is created and reviewed by SA and PM. Only after the code passes review and is merged into the main branch on GitHub does the application automatically deploy to the Test environment on Render, There is no manual deployment step is needed, ensuring rapid integration and testing of new features.

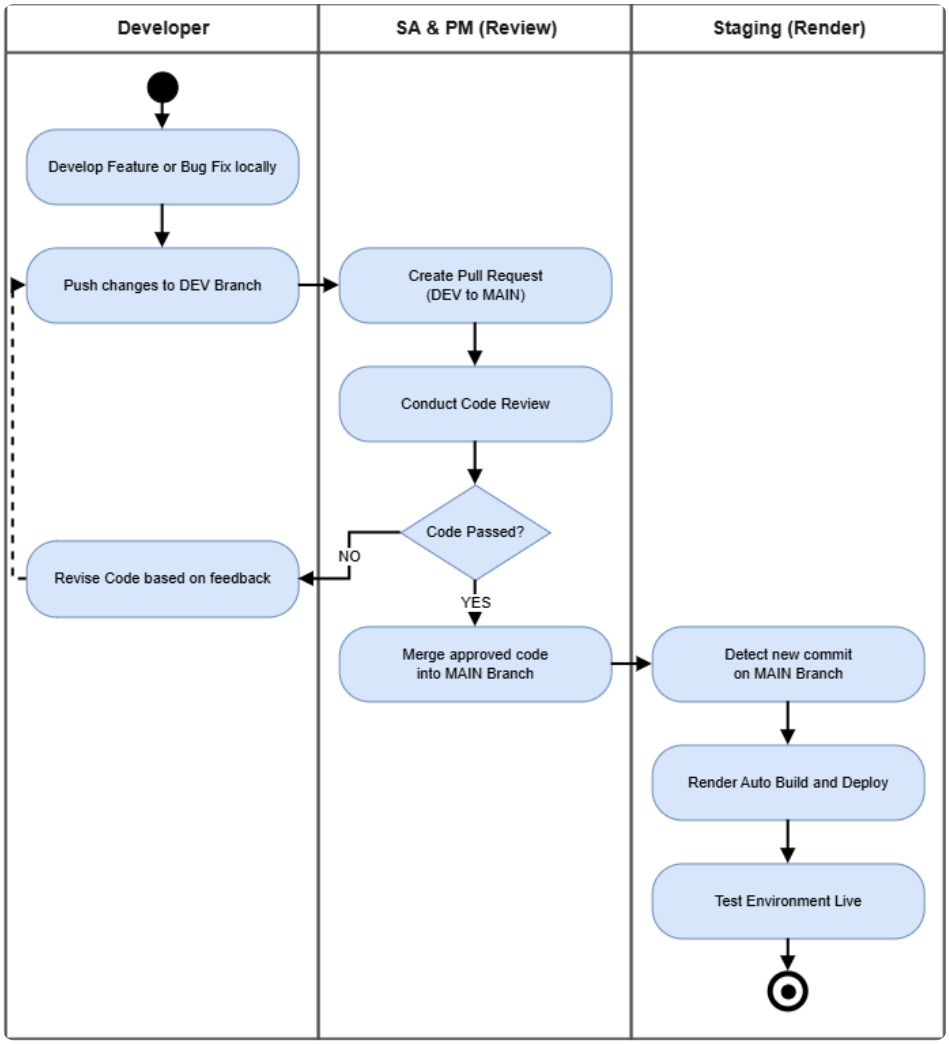


Figure 1 - Sprint Release Workflow from DEVELOPER Branch to MAIN Branch with SA and PM Review Gate and Automated Deploy to Render Test Environment.

Workflow shown in Figure 1:

Step 1 - Developer develops feature or bug fix locally.

Step 2 - Developer pushes changes to DEVELOPER Branch.

Step 3 - Create Pull Request from DEVELOPER to MAIN.

Step 4 - Conduct Code Review by SA and PM.

Step 5 - Decision Code Passed. If NO, revise code based on feedback and push again to DEV. If YES, proceed to merge.

Step 6 - Merge approved code into MAIN Branch.

Step 7 - Render detects new commit on MAIN Branch.

Step 8 - Trigger Automated Build Process via Dockerfile.

Step 9 - Deploy to Testing Environment.

FINAL RELEASE:

At the end of a sprint review or milestone:

1. Open a release pull request summarizing the tickets included.
2. Review the pre-deploy checklist.
3. Merge and tag the version in GitHub.
4. Perform a manual smoke check to verify system stability and verify core user journeys (such as inventory lookup and stock updates).
5. Announce the release to the team channel prior to demonstration.

PRE-DEPLOY CHECKLIST:

- [ ] GitHub actions or deployment pipeline runs successfully without errors.
- [ ] Database schema updates or migrations in Supabase have been reviewed. Direct manual editing of live database is prohibited.
- [ ] All required environment variables such as database connection strings are updated in Render. APP KEY, DB HOST, and DB PASSWORD must be set manually.
- [ ] No sensitive API keys or credentials are exposed in the repository.
- [ ] A local test dry-run has been completed to catch unexpected bugs.
- [ ] Vite build passes and public build assets are generated.
- [ ] pdo pgsql extension is present in production image.
- [ ] APP URL matches the deployed Render domain so links and redirects use https.

CONFIGURATION MANAGEMENT:

- Environment variables (such as DATABASE HOST, DATABASE PORT, DATABASE USERNAME, and DATABASE PASSWORD) are stored securely in the Render dashboard and not hardcoded into the source code.
- A blank .env.example file is tracked in the repository to guide team members on required configurations.
- Any configuration change must be communicated to the team to prevent unexpected system behavior.

SECRETS MANAGEMENT:

- Secrets are stored exclusively in the secure environment vault of the hosting provider (Render/GitHub).
- Only authorized team members have direct access to production and test keys.
- Secret tokens are never logged or echoed to debug outputs.

DATABASE MIGRATIONS AND BACKUP (Supabase)

DATABASE MIGRATIONS:

- All structural updates to the hardware inventory tables are managed via migration scripts.
- Direct manual editing of the live database is strictly prohibited.
- Migrations use Session Pooler port 5432 on local Globe IPv4. Transaction Pooler port 6543 is for prod runtime only and cannot be used for migrate.

BACKUP AND RESTORE:

- **What:** Automated daily snapshots managed natively by Supabase plus app-level JSON dump.
- **Retention:** 7 days for Supabase snapshots.
- **Coverage:** Complete database backup protecting all product inventory records, user accounts, supplier records, sales, and transaction logs.

MONITORING, ACCESS, AND LIMITS

MONITORING AND HEALTH:

- Basic liveness checks are performed to ensure the backend server hosted on Render is responsive and actively communicating with the Supabase database.

- Structured application logs are utilized to trace requests and debug issues safely without exposing user data.
- Nginx access logs go to stdout and error logs go to stderr for viewing in Render logs.
- Laravel log channel is stderr in production.

ACCESS, COST AND LIMITS:

- **Developers:** Full access to local environments and log viewing on Test.
- **Project Manager / Stakeholders:** Read-only access to verify system features during sprints.
- **Monthly Cost:** 0 USD for all current environments.

FREE-TIER LIMITS WORTH KNOWING BEFORE THEY BITE:

- **Render App Hosting:** The free instance will spin down (sleep) after 15 minutes of inactivity, causing a short delay on the first incoming request.
- **Supabase Database:** Limited to 1 GB of storage and strict connection pooling caps, which are more than sufficient for the hardware store inventory capstone prototype.